



Channel Capacity of Arbitrary Discrete Memoryless Channels

Detailed Deduction of Arimoto's Algorithm

Jyun-Hong Pan

National Cheng Kung University
Department of Engineering Science

June, 2026





Outline

- 1 Definitions and Problem Statement
- 2 Iterative Algorithm of Computation
- 3 Proof of Convergence
- 4 Example: 3×3 Asymmetric Channel
- 5 Summary



Symbol Definitions

Basic Symbols

Symbol	Meaning
X	Input random variable (n discrete values)
Y	Output random variable (m discrete values)
$p(x_j)$	Probability of input at x_j
$p(y_i)$	Probability of output at y_i
$p(x_j, y_i)$	Joint probability at (x_j, y_i)
$p(y_i x_j)$	Forward conditional probability (channel)
$p(x_j y_i)$	Backward conditional probability (posterior)



Entropy and Mutual Information

Entropy of Input

$$H(X) = \sum_{j=1}^n -p(x_j) \log(p(x_j))$$

Conditional Entropy $H(X | Y)$

$$H(X | Y) = \sum_{i=1}^m \sum_{j=1}^n -p(y_i | x_j) p(x_j) \log \left(\frac{p(y_i | x_j) p(x_j)}{\sum_{k=1}^n p(y_i | x_k) p(x_k)} \right)$$

Mutual Information (non-symmetric: backward / posterior form)

$$\begin{aligned} I(X; Y) &= H(X) - H(X | Y) \\ &= \sum_{i=1}^m \sum_{j=1}^n p(y_i | x_j) p(x_j) \log \frac{p(x_j | y_i)}{p(x_j)} \end{aligned}$$



Expansion of $H(X) - J(X | Y, \phi)$

Definition of $J(X | Y, \phi)$

$$J(X | Y, \phi) = \sum_{i=1}^m \sum_{j=1}^n -p(y_i | x_j) p(x_j) \log \phi(x_j | y_i)$$

$$\begin{aligned} H(X) - J(X | Y, \phi) &= \sum_{j=1}^n -p(x_j) \log p(x_j) \\ &\quad - \left(\sum_{i=1}^m \sum_{j=1}^n -p(y_i | x_j) p(x_j) \log \phi(x_j | y_i) \right) \\ &= \sum_{i=1}^m \sum_{j=1}^n p(y_i | x_j) p(x_j) \log \phi(x_j | y_i) \\ &\quad - \sum_{i=1}^m \sum_{j=1}^n p(y_i | x_j) p(x_j) \log p(x_j) \\ &= \sum_{i=1}^m \sum_{j=1}^n p(y_i | x_j) p(x_j) \log \frac{\phi(x_j | y_i)}{p(x_j)} \end{aligned}$$



Expansion of $H(X) - J(X | Y, \phi^t)$ at Iteration t

Definition of $J(X | Y, \phi^t)$

$$J(X | Y, \phi^t) = \sum_{i=1}^m \sum_{j=1}^n -p(y_i | x_j) p^t(x_j) \log \phi^t(x_j | y_i)$$

$$\begin{aligned} H(X) - J(X | Y, \phi^t) &= \sum_{j=1}^n -p^t(x_j) \log p^t(x_j) \\ &\quad - \left(\sum_{i=1}^m \sum_{j=1}^n -p(y_i | x_j) p^t(x_j) \log \phi^t(x_j | y_i) \right) \\ &= \sum_{i=1}^m \sum_{j=1}^n p(y_i | x_j) p^t(x_j) \log \phi^t(x_j | y_i) \\ &\quad - \sum_{i=1}^m \sum_{j=1}^n p(y_i | x_j) p^t(x_j) \log p^t(x_j) \\ &= \sum_{i=1}^m \sum_{j=1}^n p(y_i | x_j) p^t(x_j) \log \frac{\phi^t(x_j | y_i)}{p^t(x_j)} \end{aligned}$$



The Three-Step Iterative Algorithm

Step 1: Initialization

Set the input distribution to be uniform:

$$p^0(x_1) = p^0(x_2) = \cdots = p^0(x_n) = \frac{1}{n}$$

Let $t = 0, 1, 2, \dots$ denote the iteration index.

Step 2: Update ϕ^t and Compute $C(t, t)$

Fix $p^t(x_j)$, find the optimal ϕ^t :

$$\phi^t(x_j | y_i) = \left[\frac{p(y_i | x_j) p^t(x_j)}{\sum_{k=1}^n p(y_i | x_k) p^t(x_k)} \right]_{\forall i,j}$$



Step 2 (cont.): Compute Capacity $C(t, t)$

Capacity at iteration t

$$\begin{aligned} C(t, t) &= H(X) - H(X | Y) \\ &= \sum_{j=1}^n -p^t(x_j) \log p^t(x_j) \\ &\quad - \sum_{i=1}^m \sum_{j=1}^n -p(y_i | x_j) p^t(x_j) \log \frac{p(y_i | x_j) p^t(x_j)}{\sum_k p(y_i | x_k) p^t(x_k)} \end{aligned}$$

Interpretation

Since ϕ^t is already optimal at p^t , we get $J = H(X | Y)$. So $C(t, t)$ is the true mutual information at the current p^t .



Step 3: Update $p^{t+1}(x_j)$ via Lagrange

Now fix ϕ^t and maximize over $\{p(x_j)\}$ subject to $\sum_j p(x_j) = 1$.

Lagrangian Function

$$\begin{aligned} F = & \sum_{j=1}^n -p^t(x_j) \log p^t(x_j) \\ & - \sum_{i,j} -p(y_i | x_j) p^t(x_j) \log \phi^t(x_j | y_i) \\ & + \lambda \left(1 - \sum_{j=1}^n p^t(x_j) \right) \end{aligned}$$

Stationarity Conditions

$$\frac{\partial F}{\partial p^t(x_j)} = 0, \quad \frac{\partial F}{\partial \lambda} = 0$$



Step 3: Taking Derivatives

Compute $\partial F / \partial p^t(x_j)$:

$$\begin{aligned} \frac{\partial F}{\partial p^t(x_j)} &= -\log p^t(x_j) - p^t(x_j) \cdot \frac{1}{p^t(x_j)} \\ &\quad + \sum_{i=1}^m p(y_i | x_j) \log \phi^t(x_j | y_i) - \lambda \\ &= -\log p^t(x_j) - 1 + \sum_{i=1}^m p(y_i | x_j) \log \phi^t(x_j | y_i) - \lambda \end{aligned}$$

Setting $\partial F / \partial p^t(x_j) = 0$

$$\begin{aligned} \log p^{t+1}(x_j) &= -1 - \lambda + \sum_{i=1}^m p(y_i | x_j) \log \phi^t(x_j | y_i) \\ p^{t+1}(x_j) &= \exp \left(-1 - \lambda + \sum_{i=1}^m p(y_i | x_j) \log \phi^t(x_j | y_i) \right) \end{aligned}$$



Step 3: Solving for λ

Use the constraint $\sum_{j=1}^n p^{t+1}(x_j) = 1$:

$$\sum_{j=1}^n \exp\left(-1 - \lambda + \sum_{i=1}^m p(y_i | x_j) \log \phi^t(x_j | y_i)\right) = 1$$

$$\exp(-1 - \lambda) \cdot \sum_{j=1}^n \exp\left(\sum_{i=1}^m p(y_i | x_j) \log \phi^t(x_j | y_i)\right) = 1$$

$$\exp(-1 - \lambda) = \frac{1}{\sum_{j=1}^n \exp(\sum_{i=1}^m p(y_i | x_j) \log \phi^t(x_j | y_i))}$$



Step 3: Final Update Rule

Define

$$r_j^t = \exp \left(\sum_{i=1}^m p(y_i | x_j) \log \phi^t(x_j | y_i) \right)$$

Update Formula for $p^{t+1}(x_j)$

$$p^{t+1}(x_j) = \frac{r_j^t}{\sum_{j=1}^n r_j^t}$$

Updated Capacity (still uses ϕ^t)

$$C(t+1, t) = \log \sum_{j=1}^n r_j^t$$

Becomes $C(t+1, t+1)$ only after Step 2 of the next iteration recomputes ϕ^{t+1} .



Derivation of $C(t+1, t) = \log \sum_j r_j^t$

$$\begin{aligned}
 C(t+1, t) &= \sum_j -p^{t+1}(x_j) \log p^{t+1}(x_j) - \sum_{i,j} -p(y_i | x_j) p^{t+1}(x_j) \log \phi^t(x_j | y_i) \\
 &= \sum_j -p^{t+1}(x_j) \log p^{t+1}(x_j) + \sum_j p^{t+1}(x_j) \log r_j^t \\
 &= \sum_j p^{t+1}(x_j) \log \frac{r_j^t}{p^{t+1}(x_j)} \\
 &= \sum_j \frac{r_j^t}{\sum_k r_k^t} \cdot \log \frac{r_j^t}{r_j^t / \sum_k r_k^t} \\
 &= \frac{\sum_j r_j^t}{\sum_k r_k^t} \cdot \log \sum_j r_j^t = \log \sum_{j=1}^n r_j^t
 \end{aligned}$$



Problem Setup: A 3×3 Channel

Channel Transition Matrix $P(Y | X)$

$$P = \begin{bmatrix} p(y_1 | x_1) & p(y_1 | x_2) & p(y_1 | x_3) \\ p(y_2 | x_1) & p(y_2 | x_2) & p(y_2 | x_3) \\ p(y_3 | x_1) & p(y_3 | x_2) & p(y_3 | x_3) \end{bmatrix} = \begin{bmatrix} 0.6 & 0.7 & 0.5 \\ 0.3 & 0.1 & 0.05 \\ 0.1 & 0.2 & 0.45 \end{bmatrix}$$

This is NOT symmetric

Rows are not permutations of each other $\Rightarrow$ uniform input is **not** optimal.

Initialization

$$p^0(x_1) = p^0(x_2) = p^0(x_3) = \frac{1}{3}$$



Iteration Loop Summary

For each iteration $t = 0, 1, 2, \dots$

(1) Compute backward matrix ϕ^t :

$$\phi^t(x_j | y_i) = \frac{p(y_i | x_j) p^t(x_j)}{\sum_{k=1}^3 p(y_i | x_k) p^t(x_k)}$$

(2) Compute r_j^t :

$$r_j^t = \exp \left(\sum_{i=1}^3 p(y_i | x_j) \log \phi^t(x_j | y_i) \right)$$

(3) Update input distribution:

$$p^{t+1}(x_j) = \frac{r_j^t}{\sum_{j=1}^3 r_j^t}$$

(4) Capacity now (with the old ϕ^t):

$$C(t+1, t) = \log \sum_{j=1}^3 r_j^t$$

(5) Capacity after the next ϕ -update ϕ^{t+1} (step (1) of iteration $t+1$):

$$C(t+1, t+1) = \sum_{i,j} p(y_i | x_j) p^{t+1}(x_j) \log \frac{\phi^{t+1}(x_j | y_i)}{p^{t+1}(x_j)}$$



Iteration $t = 0$: Compute ϕ^0 and $C(0, 0)$

All logarithms natural (units: nats). $p^0 = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$.

(1) Output marginals $q(y_i) = \sum_k p(y_i | x_k) p^0(x_k)$:

$$\begin{aligned} q(y_1) &= (0.6 + 0.7 + 0.5) \frac{1}{3} = 0.6000, & q(y_2) &= (0.3 + 0.1 + 0.05) \frac{1}{3} = 0.1500, \\ q(y_3) &= (0.1 + 0.2 + 0.45) \frac{1}{3} = 0.2500. \end{aligned}$$

(2) Backward matrix $\phi^0(x_j | y_i) = \frac{p(y_i | x_j) p^0(x_j)}{q(y_i)}$, e.g. $\phi^0(x_1 | y_1) = \frac{0.6 \cdot \frac{1}{3}}{0.6} = 0.3333$:

$$\phi^0 = \begin{bmatrix} 0.3333 & 0.3889 & 0.2778 \\ 0.6667 & 0.2222 & 0.1111 \\ 0.1333 & 0.2667 & 0.6000 \end{bmatrix} \quad (\text{rows } y_i, \text{ cols } x_j)$$

(3) Capacity at p^0 :

$$C(0, 0) = \sum_{i,j} p(y_i | x_j) p^0(x_j) \ln \frac{\phi^0(x_j | y_i)}{p^0(x_j)} = 0.0858$$



Iteration $t = 0$: Update p^1 and $C(1, 0)$

(1) **Compute** $r_j^0 = \exp\left(\sum_i p(y_i | x_j) \ln \phi^0(x_j | y_i)\right)$,

e.g. $r_1^0 = \exp(0.6 \ln 0.3333 + 0.3 \ln 0.6667 + 0.1 \ln 0.1333) = \exp(-0.9823) = 0.3745$:

$$r^0 = (0.3745, 0.3410, 0.3752), \quad \sum_j r_j^0 = 1.0907.$$

(2) **Normalize** $p^1(x_j) = r_j^0 / \sum_j r_j^0$:

$$p^1 = \left(\frac{0.3745}{1.0907}, \frac{0.3410}{1.0907}, \frac{0.3752}{1.0907} \right) = (0.3433, 0.3126, 0.3440).$$

(3) **Updated capacity:**

$$C(1, 0) = \ln \sum_j r_j^0 = \ln 1.0907 = 0.0868 \quad (> C(0, 0)).$$



Iteration $t = 1$: Compute ϕ^1 and $C(1, 1)$

$$p^1 = (0.3433, 0.3126, 0.3440).$$

(1) Output marginals $q(y_i) = \sum_k p(y_i | x_k) p^1(x_k)$:

$$q(y_1) = 0.5969, \quad q(y_2) = 0.1515, \quad q(y_3) = 0.2517.$$

(2) Backward matrix $\phi^1(x_j | y_i) = \frac{p(y_i | x_j) p^1(x_j)}{q(y_i)}$, e.g.

$$\phi^1(x_1 | y_1) = \frac{0.6 \cdot 0.3433}{0.5969} = 0.3451:$$

$$\phi^1 = \begin{bmatrix} 0.3451 & 0.3667 & 0.2882 \\ 0.6800 & 0.2064 & 0.1136 \\ 0.1364 & 0.2484 & 0.6151 \end{bmatrix} \quad (\text{rows } y_i, \text{ cols } x_j)$$

(3) Capacity at p^1 : $C(1, 1) = 0.0878 \quad (> C(1, 0)).$



Iteration $t = 1$: Update p^2 and $C(2, 1)$

(1) Compute r_j^1 :

$$r^1 = (0.3855, 0.3203, 0.3869), \quad \sum_j r_j^1 = 1.0927.$$

(2) Normalize $p^2(x_j) = r_j^1 / \sum_j r_j^1$:

$$p^2 = \left(\frac{0.3855}{1.0927}, \frac{0.3203}{1.0927}, \frac{0.3869}{1.0927} \right) = (0.3528, 0.2931, 0.3541).$$

(3) Updated capacity:

$$C(2, 1) = \ln 1.0927 = 0.0887 \quad (> C(1, 1)).$$



Iteration $t = 2$: Compute ϕ^2 and $C(2, 2)$ — stop here

$$p^2 = (0.3528, 0.2931, 0.3541).$$

(1) **Output marginals** $q(y_i) = \sum_k p(y_i | x_k) p^2(x_k)$:

$$q(y_1) = 0.5939, \quad q(y_2) = 0.1529, \quad q(y_3) = 0.2533.$$

(2) **Backward matrix** $\phi^2(x_j | y_i) = \frac{p(y_i | x_j) p^2(x_j)}{q(y_i)}$, e.g.

$$\phi^2(x_1 | y_1) = \frac{0.6 \cdot 0.3528}{0.5939} = 0.3564:$$

$$\phi^2 = \begin{bmatrix} 0.3564 & 0.3455 & 0.2981 \\ 0.6924 & 0.1918 & 0.1158 \\ 0.1393 & 0.2315 & 0.6292 \end{bmatrix} \quad (\text{rows } y_i, \text{ cols } x_j)$$

(3) **Capacity at p^2 :** $C(2, 2) = 0.0896 \quad (> C(2, 1)).$

The pattern is now clear: $p(x_2)$ keeps shrinking; $p(x_1), p(x_3)$ keep growing; C rises at every half-step. The same two moves (ϕ -update, p -update) just repeat until convergence.



Summary of the First Three Iterations

Capacity increases monotonically (nats):

After computing	Capacity	Value
$\phi^0 (t = 0)$	$C(0, 0)$	0.0858
$p^1 (t = 0)$	$C(1, 0)$	0.0868
$\phi^1 (t = 1)$	$C(1, 1)$	0.0878
$p^2 (t = 1)$	$C(2, 1)$	0.0887
$\phi^2 (t = 2)$	$C(2, 2)$	0.0896

The rule the audience can now see:

$$C(0, 0) \leq C(1, 0) \leq C(1, 1) \leq C(2, 1) \leq C(2, 2) \leq \dots$$

The ϕ -update (optimize ϕ) and the p -update (optimize p) each never decrease C , so the algorithm climbs steadily to capacity (full convergence table follows).



Stopping Criterion

Convergence Test

Stop iterating when

$$|C(t+1, t+1) - C(t, t)| < \varepsilon$$

where $\varepsilon = 10^{-6}$ (project requirement).

Note

- Convergence is guaranteed but can be slow.
- Each iteration is $O(nm)$ in operations.
- Far more efficient than direct optimization over the $(n-1)$ -dim simplex.



Numerical Results: Convergence Table

t	$C(t, t)$ [nats]	$p(x_1)$	$p(x_2)$	$p(x_3)$
1	0.087757	0.34332	0.31265	0.34404
10	0.099720	0.41194	0.17337	0.41469
20	0.105998	0.45461	0.089962	0.45543
30	0.108968	0.47696	0.047113	0.47593
40	0.110442	0.48865	0.024857	0.4865
50	0.111198	0.49479	0.013176	0.49203
60	0.111592	0.49805	0.0070022	0.49495
70	0.111800	0.49977	0.0037268	0.4965
80	0.111910	0.50069	0.001985	0.49733
90	0.111968	0.50118	0.0010578	0.49776
100	0.111999	0.50144	0.00056377	0.498
110	0.112016	0.50158	0.00030052	0.49812
115	0.112021	0.50162	0.00021942	0.49816



Converged Result

Final Optimal Input Distribution

$$p(x_1) = 0.50162, \quad p(x_2) = 2.19 \times 10^{-4}, \quad p(x_3) = 0.49816$$

Channel Capacity

$$C = 0.11202 \text{ nats/use} = 0.16161 \text{ bits/use}$$

Key Observation

- Optimal $p(x_2) \approx 0$: symbol x_2 should **almost never be used!**
- Symbols x_1 and x_3 share the input mass nearly equally.
- A ternary input can convey at most $\log 3 = 1.099$ nats (1.585 bits), and only over a noiseless channel; this noisy, asymmetric channel reaches just 0.112 nats.



Why is $p(x_2) \approx 0$?

Inspect Column 2 of $P(Y | X)$

$$P(Y | x_2) = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.2 \end{bmatrix}$$

Compare to $P(Y | x_1) = (0.6, 0.3, 0.1)^\top$ and $P(Y | x_3) = (0.5, 0.05, 0.45)^\top$.

Insight

- Output of x_2 is statistically very similar to a mixture of x_1 and x_3 .
- Using x_2 provides little extra information beyond what x_1, x_3 already give.
- Optimal coding strategy: **abandon the redundant symbol**.



Summary of Arimoto's Algorithm

Core Insight

Two-variable joint optimization $\Rightarrow$ alternating maximization

- Inner: $\phi^t(x_j | y_i) = \frac{p(y_i | x_j) p^t(x_j)}{\sum_k p(y_i | x_k) p^t(x_k)}$
- Outer: $p^{t+1}(x_j) = \frac{r_j^t}{\sum_j r_j^t}$

Key Properties

- Monotonic increase: $C(t, t) \nearrow C(P)$
- Provably convergent to global optimum (convexity of mutual information in $p(x)$ for fixed $p(y | x)$)
- Simple to implement, only matrix-vector operations



Recap of Important Formulas

The 4 Essential Equations

$$\phi^t(x_j | y_i) = \frac{p(y_i | x_j) p^t(x_j)}{\sum_k p(y_i | x_k) p^t(x_k)}$$

$$r_j^t = \exp\left(\sum_{i=1}^m p(y_i | x_j) \log \phi^t(x_j | y_i)\right)$$

$$p^{t+1}(x_j) = \frac{r_j^t}{\sum_j r_j^t}$$

$$C(t+1, t) = \log \sum_{j=1}^n r_j^t$$



References



S. Arimoto, “An algorithm for computing the capacity of arbitrary discrete memoryless channels,” *IEEE Trans. Inf. Theory*, vol. 18, no. 1, pp. 14–20, 1972.



C. Xiong, L. Pei, T.-K. Truong, “Detailed Deduction on Arimoto’s Channel Capacity Computation.”



T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006.



Appendix: The Coupling Problem

Expanding $I(X; Y) = H(X) - H(X | Y)$

Starting from

$$I(X; Y) = \sum_{x,y} p(x) p(y | x) \log \frac{p(x | y)}{p(x)},$$

substitute Bayes' rule $p(x | y) = \frac{p(x) p(y | x)}{\sum_{x'} p(x') p(y | x')}$:

$$I(X; Y) = \sum_{x,y} p(x) p(y | x) \log \frac{p(y | x)}{\sum_{x'} p(x') p(y | x')}.$$

Source of Coupling

The denominator $\sum_{x'} p(x') p(y | x')$ **locks all input probabilities inside the same log**: differentiating w.r.t. any single $p(x_j)$ yields a nonlinear system — no closed-form solution by direct optimization.



Appendix: Variational Lower Bound via $\phi(x | y)$

Introduce an Auxiliary Distribution $\phi(x | y)$

Consider the gap between the true mutual information and a ϕ -based surrogate:

$$I(X; Y) - \sum_{x,y} p(x) p(y | x) \log \frac{\phi(x | y)}{p(x)} = \sum_{x,y} p(x) p(y | x) \log \frac{p(x | y)}{\phi(x | y)}$$

The RHS is a **conditional KL divergence** ≥ 0 , equality iff $\phi(x | y) = p(x | y)$.

Variational Form of Mutual Information

$$I(X; Y) = \max_{\phi} \sum_{x,y} p(x) p(y | x) \log \frac{\phi(x | y)}{p(x)}$$



Appendix: How Decoupling Happens

Fixing $\phi(x | y)$, Maximizing over $p(x)$

With ϕ held fixed, the term $\frac{\phi(x | y)}{p(x)}$ depends on each $p(x_j)$ **independently**.

- The troublesome $\sum_{x'}$ inside the log is **gone**.
- The stationary condition $\partial/\partial p(x_j) = 0$ separates into one equation per j .
- $\Rightarrow$ closed-form update $p^{t+1}(x_j) = r_j^t / \sum_k r_k^t$.

Geometric Interpretation (Lagrange)

At the optimum, the gradient of the objective is parallel to the gradient of the simplex constraint $\sum_j p(x_j) = 1$. Alternating max over (p, ϕ) traces a path of monotonic improvement along this constrained manifold.



Appendix: Proof of the Log-Sum Inequality

Statement

For nonnegative reals $a_1, \dots, a_n$ and $b_1, \dots, b_n$ (with $b_i > 0$):

$$\sum_{i=1}^n a_i \log \frac{a_i}{b_i} \geq \left(\sum_{i=1}^n a_i \right) \log \frac{\sum_{i=1}^n a_i}{\sum_{i=1}^n b_i}$$

with equality iff a_i/b_i is constant in i .

Proof via Jensen's Inequality

Let $A = \sum_i a_i$, $B = \sum_i b_i$, and define weights $w_i = b_i/B$ (so $\sum_i w_i = 1$). Since $f(t) = t \log t$ is convex on $(0, \infty)$, by Jensen's inequality:

$$\sum_i w_i f\left(\frac{a_i}{b_i}\right) \geq f\left(\sum_i w_i \cdot \frac{a_i}{b_i}\right) = f\left(\frac{A}{B}\right).$$



Appendix: Proof of the Log-Sum Inequality (cont.)

Expand Both Sides

$$\text{LHS: } \sum_i \frac{b_i}{B} \cdot \frac{a_i}{b_i} \log \frac{a_i}{b_i} = \frac{1}{B} \sum_i a_i \log \frac{a_i}{b_i}.$$

$$\text{RHS: } \frac{A}{B} \log \frac{A}{B}.$$

Multiplying both sides by $B > 0$:

$$\sum_i a_i \log \frac{a_i}{b_i} \geq A \log \frac{A}{B} = \left(\sum_i a_i \right) \log \frac{\sum_i a_i}{\sum_i b_i}.$$

Equality Condition

Jensen's inequality is tight iff the convex function is evaluated at a single point, i.e. a_i/b_i is identical for all i with $w_i > 0$.



Thank You!

Questions?